



Video Summary

This Telusko lesson introduces functions as named blocks of code that make programs cleaner, modular, and reusable. It moves from basic scripting toward structured Python using VS Code, parameters, return values, and docstrings.

Source: [user-provided Telusko/Navin Reddy Day 16 video summary](#)

1

Why Functions Matter

Functions package a task under a clear name. Once written, the same logic can be called many times without copying the same statements again.

()

Reusable

Write the logic once, then call it whenever the same task is needed.

def

Modular

Break a bigger program into smaller named tasks that are easier to test.

{ }

Cleaner

Keep the main program readable by moving details into functions.

```
>>> def greet():
...     print("Hello, Python learner")
...
>>> greet()
Hello, Python learner
>>> greet()
Hello, Python learner
```

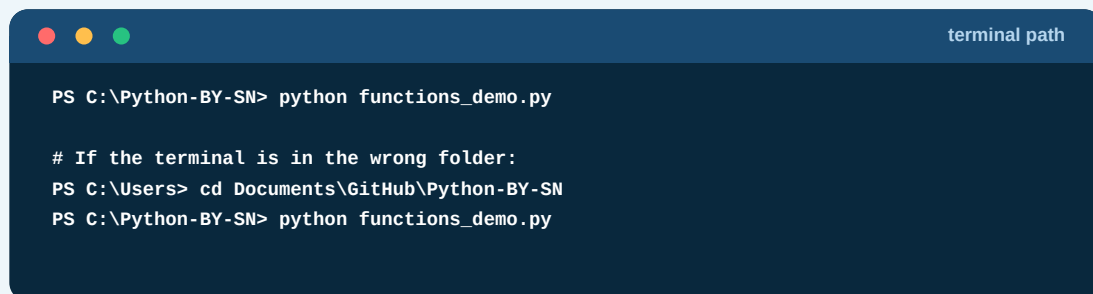
first function

2

Set Up the Workspace

The lesson begins by preparing the coding environment in VS Code. A clean workspace and terminal path make it easier to run files without confusion.

Step	What to do	Why it helps
Open folder	Open the lesson folder in VS Code.	Keeps files in one visible project area.
Create file	Make a Python file such as functions_demo.py.	The .py file can be run from the terminal.
Check path	Use the integrated terminal in that folder.	Commands run against the correct file location.
Keep clean	Avoid jumping across unrelated directories.	Fewer path errors while practicing functions.



```
terminal path

PS C:\Python-BY-SN> python functions_demo.py

# If the terminal is in the wrong folder:
PS C:\Users> cd Documents\GitHub\Python-BY-SN
PS C:\Python-BY-SN> python functions_demo.py
```

Setup habit

Before writing function code, confirm that VS Code is opened in the correct folder and the terminal path points to the same project.

3

Defining and Calling Functions

A function definition starts with `def`, followed by the function name, round brackets, and a colon. The indented lines below it form the function body.

Part	Example	Meaning
def	<code>def say_hello():</code>	Starts a function definition.
Name	<code>say_hello</code>	The label used to call the function.
Round brackets	<code>()</code>	Holds parameters when input is needed.
Colon	<code>:</code>	Marks the start of the function block.
Indentation	<code>print("Hi")</code>	Shows which statements belong to the function.

```
function structure

def say_hello():
    print("Hello")
    print("Welcome to functions")

# The function runs only when called.
say_hello()
say_hello()
```

Indentation rule

Python does not use curly braces for function scope. Use consistent indentation, commonly four spaces, for every line inside the function.

4

Make Functions Flexible

Instead of hardcoding values inside a function, define parameters in the brackets and pass arguments when calling the function. This makes one function work for many inputs.

x

Parameter

A variable name written in the function definition. It receives the incoming value.

5

Argument

The actual value supplied when the function is called.

+

Body

The indented code that uses the parameters to perform the task.

```
def add(a, b):  
    result = a + b  
    print(result)  
  
add(4, 5)  
add(10, 20)  
  
Output:  
9  
30
```

Beginner vocabulary

In `def add(a, b)`, `a` and `b` are parameters. In `add(4, 5)`, `4` and `5` are arguments passed into those parameters.

5

Use the return Statement

Printing shows a value on the screen. Returning sends a value back to the main program, so it can be stored, printed later, or used in another calculation.

Approach	What happens	Best use
<code>print()</code>	Displays output immediately inside the function.	Quick messages and debugging.
<code>return</code>	Hands data back to the caller.	Calculations and reusable logic.
No <code>return</code>	Function returns <code>None</code> by default.	Action-only tasks.

return value example

```
def add(a, b):  
    return a + b  
  
answer = add(4, 5)  
print(answer)  
print(answer * 2)  
  
Output:  
9  
18  
  
def show_sum(a, b):  
    print(a + b) # returns None
```

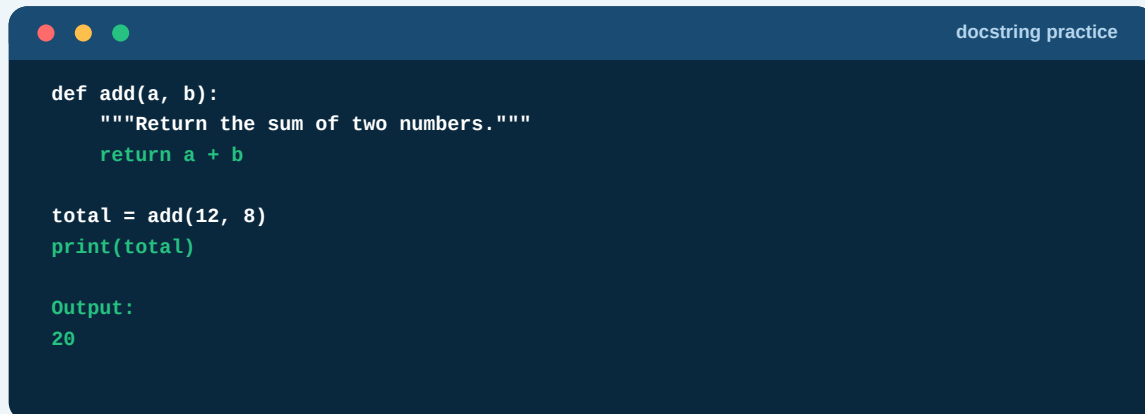
Print versus return

Use `print()` when you only want to display something. Use `return` when the main program needs the value for later work.

6

Document the Function

A docstring is a short description placed inside triple double quotes at the top of a function body. Editors can show it as help when you hover over or inspect the function.



```
def add(a, b):  
    """Return the sum of two numbers."""  
    return a + b  
  
total = add(12, 8)  
print(total)  
  
Output:  
20
```

Checklist	Question to ask	Good answer
Name	Does the function name describe the task?	Use names like add, greet, or calculate_total.
Input	Should values come from parameters?	Use parameters instead of hardcoded numbers.
Output	Should the result be reused?	Use return when the caller needs the value.
Docs	Will the purpose be clear later?	Add a concise triple-quoted docstring.

Day 16/55 takeaway

Functions turn repeated statements into named, reusable tasks. Learn def, indentation, calls, parameters, return, and docstrings before writing larger programs.